CC3067 Redes · Laboratorio de IoT

Estación meteorológica con Apache Kafka

Simulamos un nodo con tres sensores, mandamos su telemetría a un broker Kafka en el Edge y la graficamos en vivo. Después metimos cada lectura en 3 bytes.

Esteban Cárcamo · 23016
Felipe Aguilar · 23195

12 de noviembre de 2026 · github.com/ecarcamo/IoT-Estacion-Meteorologica

Panorama

Cómo quedó armado



- Todo está en Python con `kafka-python`. Como topic usamos el carné de Esteban.
- El productor hace de "dispositivo IoT"; el consumidor es lo que estaría detrás del Edge.
- La contraseña SASL vive en un `.env` que no subimos al repositorio.
- En el topic conviven mensajes JSON y de 3 bytes. El consumidor los distingue por el largo: si mide 3 bytes, es compacto.

Tres sensores con números aleatorios

Sensor	Cómo lo generamos	Rango y tipo
Temperatura	Normal con μ = 55 y σ = 15	0 a 110 °C, dos decimales
Humedad relativa	La misma normal, redondeada	0 a 100 %, entero
Dirección del viento	Uniforme entre las 8 opciones	N, NO, O, SO, S, SE, E, NE

La varianza quedaba a nuestro criterio y escogimos σ = 15 (varianza 225). Con eso casi todo cae entre 10 y 100 (μ ± 3σ), así que el recorte a los límites del sensor casi nunca entra en juego, y aun así la gráfica sube y baja lo suficiente para que se note.

Cada lectura sale como JSON:

```
{"temperatura": 56.32, "humedad": 51, "direccion_viento": "SO"}
```

3.1 · Evidencia

Lecturas generadas y pruebas

Lecturas sacadas de `WeatherSensor`, cada una con su versión de 3 bytes a la derecha. Abajo corren las 20 pruebas de pytest que revisan rangos, codificación y decodificación.

```
$ python -c "from weather_station.sensors import WeatherSensor; ..."
{"temperatura": 84.78, "humedad": 90, "direccion_viento": "SE"} -> 84 7a d5
{"temperatura": 61.95, "humedad": 56, "direccion_viento": "O"} -> 60 cd c2
{"temperatura": 40.26, "humedad": 52, "direccion_viento": "SO"} -> 3e e9 a3
{"temperatura": 74.65, "humedad": 36, "direccion_viento": "E"} -> 74 a5 26
{"temperatura": 49.28, "humedad": 70, "direccion_viento": "NO"} -> 4d 02 31
{"temperatura": 49.63, "humedad": 48, "direccion_viento": "N"} -> 4d 8d 80
{"temperatura": 76.47, "humedad": 64, "direccion_viento": "NO"} -> 77 7e 01
{"temperatura": 57.54, "humedad": 73, "direccion_viento": "O"} -> 59 ea 4a

$ pytest -v
tests/test_codec.py::test_example_from_assignment PASSED                [  5%]
tests/test_codec.py::test_extremes_fit_in_three_bytes PASSED            [ 10%]
tests/test_codec.py::test_every_simulated_reading_is_three_bytes PASSED  [ 15%]
tests/test_codec.py::test_decode_rejects_wrong_size[] PASSED            [ 20%]
tests/test_codec.py::test_decode_rejects_wrong_size[\x00\x00] PASSED     [ 25%]
tests/test_codec.py::test_decode_rejects_wrong_size[\x00\x00\x00\x00] PASSED [ 30%]
tests/test_codec.py::test_rejects_out_of_range_values[data0] PASSED      [ 35%]
tests/test_codec.py::test_rejects_out_of_range_values[data1] PASSED      [ 40%]
tests/test_codec.py::test_rejects_out_of_range_values[data2] PASSED      [ 45%]
tests/test_consumer.py::test_reads_mixed_formats PASSED                  [ 50%]
tests/test_consumer.py::test_rejects_invalid_readings[None] PASSED       [ 55%]
tests/test_consumer.py::test_rejects_invalid_readings[] PASSED           [ 60%]
tests/test_consumer.py::test_rejects_invalid_readings[[]] PASSED         [ 65%]
tests/test_consumer.py::test_rejects_invalid_readings[{}] PASSED         [ 70%]
tests/test_consumer.py::test_rejects_invalid_readings[null] PASSED       [ 75%]
tests/test_consumer.py::test_rejects_invalid_readings[\xff\xff\xff] PASSED [ 80%]
tests/test_consumer.py::test_rejects_invalid_readings[{"temperatura": NaN}] PASSED [ 85%]
tests/test_sensors.py::test_readings_stay_in_range PASSED                [ 90%]
tests/test_sensors.py::test_readings_are_centered_on_55 PASSED           [ 95%]
tests/test_sensors.py::test_json_uses_agreed_keys PASSED                 [100%]
============================== 20 passed in 0.62s ==============================
```

¿A qué capa pertenece JSON/SOAP según el modelo OSI y por qué?

A la capa 6, presentación. JSON no abre conexiones ni mueve paquetes; lo único que define es cómo se escriben los datos (llaves, tipos, texto en UTF-8) para que nuestro productor en Python y cualquier consumidor, en el lenguaje que sea, los lean igual. Esa traducción a un formato común es el trabajo de la capa de presentación. SOAP es más mixto: su XML es representación, pero como protocolo de mensajes sobre HTTP también se suele ubicar en aplicación (capa 7).

¿Qué beneficios tiene utilizar un formato como JSON/SOAP?

Lo notamos al depurar: en la terminal se lee directo qué mandó el sensor. Casi cualquier lenguaje lo trae incluido y se le pueden agregar campos sin romper a quien ya lo consume. El precio es el tamaño: una lectura nuestra pesa 62 o 63 bytes en JSON, contra 3 en el formato compacto.

El productor

`producer.py` se conecta a `161.35.11.141:9092` con `SASL_PLAINTEXT` y `PLAIN`, y publica en el topic 23016. Entre una lectura y otra espera un tiempo al azar de 15 a 30 s, y sigue así hasta que alguien lo detiene con Ctrl+C.

```
producer = KafkaProducer(**settings.client_options(), acks="all")
while args.count == 0 or sent < args.count:
    payload = encode_reading(sensor.read(), args.compact)
    metadata = producer.send(topic, key=b"sensor1", value=payload).get(timeout=30)
    time.sleep(random.uniform(15, 30))
```

- Con `acks="all"` y el `.get()`, cada envío espera la confirmación del broker. Por eso podemos imprimir la partición y el offset de cada mensaje.
- Todas las lecturas llevan la key `sensor1`, así que caen en la misma partición y no se desordenan.

3.2 · Evidencia

El productor publicando en el broker

Corrida del 10 de noviembre. Primero cinco lecturas en JSON (`python producer.py`) y luego cinco en 3 bytes (`--compact`). Por la hora de cada línea se ve que entre envíos pasan de 15 a 30 s, y al final de cada una está la partición y el offset que devolvió Kafka.

```
Enviando al topic 23016 en 161.35.11.141:9092, modo JSON. Ctrl+C para detener.
[20:55:06] #1 {"temperatura": 39.29, "humedad": 51, "direccion_viento": "N"} -> JSON (62 bytes, partición 1, offset 20)
[20:55:24] #2 {"temperatura": 38.8, "humedad": 69, "direccion_viento": "NE"} -> JSON (62 bytes, partición 1, offset 21)
[20:55:48] #3 {"temperatura": 51.03, "humedad": 69, "direccion_viento": "N"} -> JSON (62 bytes, partición 1, offset 22)
[20:56:06] #4 {"temperatura": 37.86, "humedad": 71, "direccion_viento": "S"} -> JSON (62 bytes, partición 1, offset 23)
[20:56:36] #5 {"temperatura": 91.43, "humedad": 74, "direccion_viento": "O"} -> JSON (62 bytes, partición 1, offset 24)
Enviando al topic 23016 en 161.35.11.141:9092, modo compacto (3 bytes). Ctrl+C para detener.
[20:56:37] #1 {"temperatura": 40.54, "humedad": 57, "direccion_viento": "NE"} -> 3f 59 cf (3 bytes, partición 1, offset 25)
[20:57:07] #2 {"temperatura": 27.39, "humedad": 49, "direccion_viento": "N"} -> 2a cd 88 (3 bytes, partición 1, offset 26)
[20:57:28] #3 {"temperatura": 47.01, "humedad": 43, "direccion_viento": "E"} -> 49 75 5e (3 bytes, partición 1, offset 27)
[20:57:56] #4 {"temperatura": 61.03, "humedad": 65, "direccion_viento": "NO"} -> 5f 5e 09 (3 bytes, partición 1, offset 28)
[20:58:25] #5 {"temperatura": 56.24, "humedad": 39, "direccion_viento": "S"} -> 57 e1 3c (3 bytes, partición 1, offset 29)
```

El consumidor y la gráfica

`consumer.py` se suscribe al topic 23016 con las mismas credenciales. Con cada mensaje que llega: lo decodifica (JSON o 3 bytes), revisa que los valores estén dentro de rango y lo agrega a la gráfica. Si algo viene mal, lo reporta en consola y lo salta.

La gráfica tiene tres paneles: temperatura y humedad contra la hora del mensaje, y cuántas veces ha salido cada dirección del viento. Se redibuja cada vez que entra un dato.

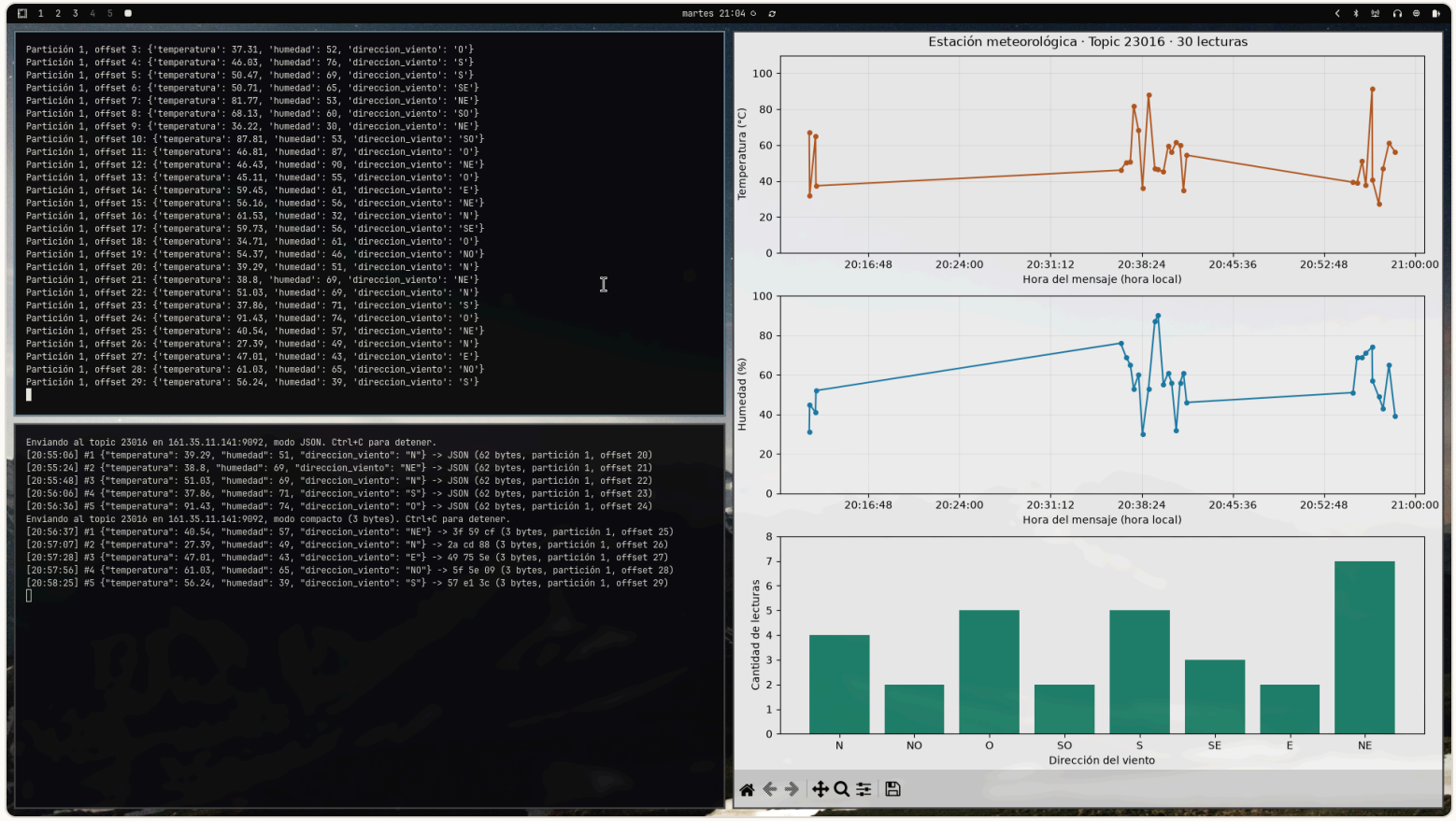
Un detalle práctico: si el programa se queda bloqueado esperando mensajes, la ventana de matplotlib se congela. Por eso usamos `poll()` con 100 ms de espera y `plt.pause()`, y la ventana sigue respondiendo entre lectura y lectura.

Con `auto_offset_reset="earliest"` también aparece lo que ya estaba en el topic, y `--replay` relee todo el historial sin mover los offsets del grupo.

3.3 · Evidencia

Productor, consumidor y gráfica a la vez

Todo corriendo contra el broker real. Arriba a la izquierda está el consumidor y abajo el productor; los offsets de uno y otro coinciden, y la gráfica de la derecha crece con cada lectura.



3.3 · Evidencia

Gráfica con datos consumidos de Kafka

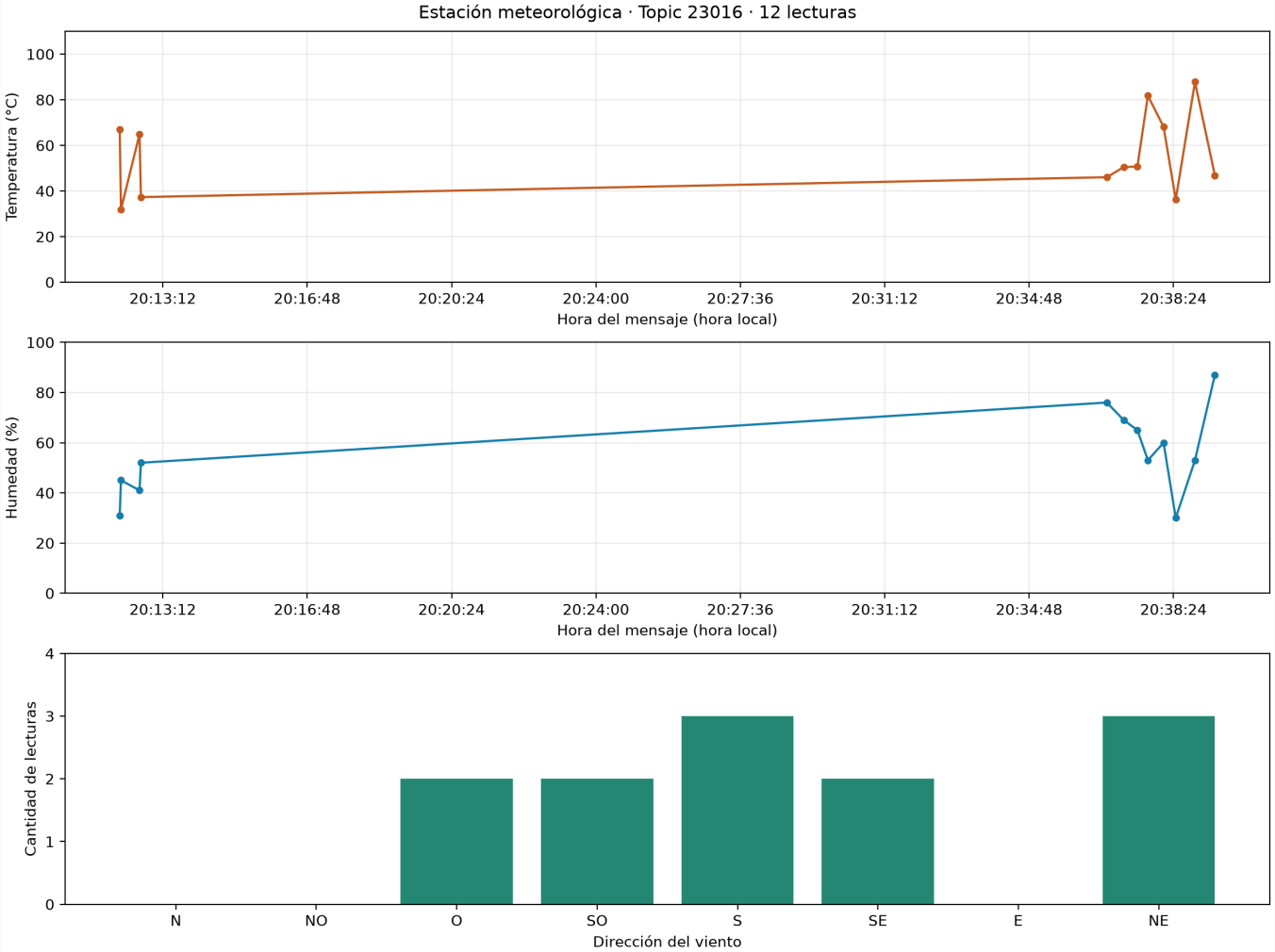


Figura exportada por el consumidor con las lecturas JSON del topic 23016 (offsets 4 a 11 más lo que ya había).

Arriba la temperatura, en medio la humedad y abajo el conteo de direcciones del viento.

El tramo recto del inicio une unas lecturas de prueba de las 20:12 con la corrida de las 20:36, porque el eje X usa la hora real de cada mensaje.

¿Qué ventajas y desventajas tiene este acercamiento basado en Pub/Sub de Kafka?

A favor

- El sensor no sabe quién lo lee. Si mañana queremos un consumidor que mande alertas, se suscribe al topic y el productor ni se entera.
- Kafka guarda los mensajes. Así pudimos reabrir la gráfica con `--replay` y ver lecturas de horas antes.
- Si el consumidor se cae, al volver sigue desde su último offset.

En contra

- Alguien tiene que instalar y mantener el broker. Aquí nos lo dieron listo, pero en un proyecto real es trabajo aparte.
- Para un sensor y una gráfica es bastante infraestructura; un socket directo haría lo mismo.
- El orden solo se garantiza dentro de una partición, y sin entender grupos y offsets es fácil leer repetido o saltarse datos.

¿Para qué aplicaciones tiene sentido usar Kafka? ¿Para cuáles no?

Sí tiene sentido

- Muchos dispositivos mandando telemetría al mismo tiempo, como este caso pero con cientos de estaciones.
- Logs de muchos servidores juntos en un solo lugar.
- Eventos que varios sistemas tienen que procesar: pedidos, pagos, clics.
- Análisis en streaming, o cuando conviene poder releer el historial.

Mejor no

- Una app pequeña con poco tráfico.
- Colas de tareas sencillas, donde RabbitMQ o Redis bastan.
- Petición y respuesta inmediata, como una API normal.
- Un sensor con batería y poca memoria hablando directo con Kafka. Ahí encaja MQTT o CoAP, con un gateway que pase los datos a Kafka.

Una lectura en 24 bits

La pista del enunciado: 2^{14} = 16 384. La temperatura máxima es 110.00, y si le quitamos el punto queda 11 000, que es menor. O sea, la multiplicamos por 100, la guardamos como entero y no perdemos ningún decimal.

Ejemplo: **56.32 °C → 5632**, **51 %**, **SO → 3**




```
(5632 << 10) | (51 << 3) | 3   =   0x58019B   →   bytes 58 01 9B
```

Encode y decode

```
def encode(data: dict) -> bytes:
    temperature = round(data["temperatura"] * TEMPERATURE_SCALE)
    humidity = data["humedad"]
    wind = WIND_DIRECTIONS.index(data["direccion_viento"])
    packed = (temperature << TEMPERATURE_SHIFT) | (humidity << HUMIDITY_SHIFT) | wind
    return packed.to_bytes(PAYLOAD_SIZE, "big")

def decode(payload: bytes) -> dict:
    packed = int.from_bytes(payload, "big")
    return {
        "temperatura": ((packed >> TEMPERATURE_SHIFT) & TEMPERATURE_MASK) / TEMPERATURE_SCALE,
        "humedad": (packed >> HUMIDITY_SHIFT) & HUMIDITY_MASK,
        "direccion_viento": WIND_DIRECTIONS[packed & WIND_MASK],
    }
```

Son los mismos corrimientos de la lámina anterior, de ida y de vuelta. Le quitamos aquí las validaciones para que quepa: en el código, `encode` rechaza valores fuera de rango, porque una humedad de 200 no cabe en 7 bits y se metería en los bits de la temperatura.

3.4 · Evidencia

Lo que sale es lo que llega

Los offsets 25 a 29 salieron del productor en 3 bytes y el consumidor los decodificó con los mismos valores. Por ejemplo, 40.54 °C, 57 % y NE viajó como `3f 59 cf`.

Productor

```
Enviando al topic 23016 en 161.35.11.141:9092, modo JSON. Ctrl+C para detener.
[20:55:06] #1 {"temperatura": 39.29, "humedad": 51, "direccion_viento": "N"} -> JSON (62 bytes, partición 1, offset 20)
[20:55:24] #2 {"temperatura": 38.8, "humedad": 69, "direccion_viento": "NE"} -> JSON (62 bytes, partición 1, offset 21)
[20:55:48] #3 {"temperatura": 51.03, "humedad": 69, "direccion_viento": "N"} -> JSON (62 bytes, partición 1, offset 22)
[20:56:06] #4 {"temperatura": 37.86, "humedad": 71, "direccion_viento": "S"} -> JSON (62 bytes, partición 1, offset 23)
[20:56:36] #5 {"temperatura": 91.43, "humedad": 74, "direccion_viento": "O"} -> JSON (62 bytes, partición 1, offset 24)
Enviando al topic 23016 en 161.35.11.141:9092, modo compacto (3 bytes). Ctrl+C para detener.
[20:56:37] #1 {"temperatura": 40.54, "humedad": 57, "direccion_viento": "NE"} -> 3f 59 cf (3 bytes, partición 1, offset 25)
[20:57:07] #2 {"temperatura": 27.39, "humedad": 49, "direccion_viento": "N"} -> 2a cd 88 (3 bytes, partición 1, offset 26)
[20:57:28] #3 {"temperatura": 47.01, "humedad": 43, "direccion_viento": "E"} -> 49 75 5e (3 bytes, partición 1, offset 27)
[20:57:56] #4 {"temperatura": 61.03, "humedad": 65, "direccion_viento": "NO"} -> 5f 5e 09 (3 bytes, partición 1, offset 28)
[20:58:25] #5 {"temperatura": 56.24, "humedad": 39, "direccion_viento": "S"} -> 57 e1 3c (3 bytes, partición 1, offset 29)
```

Consumidor, offsets 20 a 29

```
Partición 1, offset 20: {'temperatura': 39.29, 'humedad': 51, 'direccion_viento': 'N'}
Partición 1, offset 21: {'temperatura': 38.8, 'humedad': 69, 'direccion_viento': 'NE'}
Partición 1, offset 22: {'temperatura': 51.03, 'humedad': 69, 'direccion_viento': 'N'}
Partición 1, offset 23: {'temperatura': 37.86, 'humedad': 71, 'direccion_viento': 'S'}
Partición 1, offset 24: {'temperatura': 91.43, 'humedad': 74, 'direccion_viento': 'O'}
Partición 1, offset 25: {'temperatura': 40.54, 'humedad': 57, 'direccion_viento': 'NE'}
Partición 1, offset 26: {'temperatura': 27.39, 'humedad': 49, 'direccion_viento': 'N'}
Partición 1, offset 27: {'temperatura': 47.01, 'humedad': 43, 'direccion_viento': 'E'}
Partición 1, offset 28: {'temperatura': 61.03, 'humedad': 65, 'direccion_viento': 'NO'}
Partición 1, offset 29: {'temperatura': 56.24, 'humedad': 39, 'direccion_viento': 'S'}
```

La gráfica con mensajes de 3 bytes



Las lecturas compactas (offsets 12 a 19) se grafican junto al historial JSON, en el mismo topic y con el mismo consumidor.

El offset 12, por ejemplo, llegó como `48 8E D7` y se dibujó como 46.43 °C, 90 % y NE.

Desde la gráfica no se nota qué formato traía cada punto, que era justo la idea.

¿Qué complejidades introduce el tener un payload restringido?

La principal es que el mensaje deja de explicarse solo. `48 8E D7` no dice nada si no se sabe qué bits son de qué campo. Productor y consumidor tienen que usar exactamente el mismo esquema, y si uno cambia (otro orden, otro rango) hay que actualizar los dos a la vez.

Tampoco queda espacio para nada más: ni id del sensor, ni timestamp, ni versión del formato, ni checksum. Si un bit se voltea en el camino, llega otro valor y nadie se da cuenta.

Y hay que validar antes de codificar, porque un valor fuera de rango no da error al empaquetarse: se desborda hacia el campo de al lado.

¿Cómo podemos hacer que el valor de temperatura quepa en 14 bits?

Guardándola como entero en centésimas (punto fijo). 110.00 × 100 = 11 000, y en 14 bits caben hasta 16 383, así que entra todo el rango con sus dos decimales. Al recibir se divide entre 100. No hace falta un float de 4 bytes porque ya conocemos el rango y la precisión.

¿Qué sucedería si la humedad también fuera float con un decimal? ¿Qué decisiones tendríamos que tomar?

La humedad pasaría de 101 valores posibles a 1 001 (de 0.0 a 100.0, o sea 0 a 1000 en décimas), y eso pide 10 bits. Sumando: 14 + 10 + 3 = 27 bits, y solo tenemos 24. Algo hay que sacrificar:

Opción	Bits	Qué perdemos
Temperatura con un decimal (0 a 1100)	`11 + 10 + 3 = 24`	la centésima de grado
Dejar la humedad entera	`14 + 7 + 3 = 24`	el decimal nuevo de humedad
Alternar mensajes de temperatura y de humedad	`3 bytes cada uno`	la mitad de lecturas por variable
Pedir un payload de 4 bytes	`32`	ya no cumple la restricción

Nosotros escogeríamos la primera. Para una estación meteorológica, un decimal de temperatura sobra; la segunda cifra decimal es más ruido del sensor que información.

¿Qué parámetros o herramientas de Kafka podrían ayudarnos si las restricciones fueran aún más fuertes?

- `compression.type` (gzip, snappy, lz4, zstd) junto con `batch.size` y `linger.ms`: en lugar de mandar cada lectura sola, se juntan varias y se comprimen. Se gana espacio y se pierde inmediatez.
- La key. Hoy `sensor1` pesa 7 bytes, más del doble que el payload. Una key de un byte, o ninguna, ahorra eso; sin key se pierde el orden garantizado por sensor.
- `max.request.size` y `message.max.bytes` para ponerle techo al tamaño de los mensajes.
- Avro o Protobuf con Schema Registry para versionar el formato compacto sin romper consumidores.

Siendo honestos, el protocolo de Kafka agrega decenas de bytes de encabezado a cada mensaje. En un enlace LoRa real el sensor no hablaría Kafka: mandaría sus 3 bytes por radio a un gateway, y ese gateway (o Kafka Connect) los publicaría en el broker.

Cierre

Lo que nos llevamos

- Prueba contra el broker real el 10 de noviembre: 16 mensajes (8 JSON y 8 de 3 bytes, offsets 4 a 19), y todos llegaron con los mismos valores que se enviaron.
- Pasamos de unos 63 bytes a 3 (95 % menos) por lectura. Codificar fue lo fácil; lo que hay que cuidar es que los dos lados usen el mismo esquema.
- En la misma corrida pasamos de JSON a 3 bytes y el consumidor leyó ambos formatos del mismo topic. Ahí se ve bien la utilidad de Pub/Sub.
- Para un solo sensor Kafka es mucho. Empieza a valer la pena con muchos dispositivos o con varios sistemas leyendo los mismos datos.

Referencias: documentación de Apache Kafka (kafka.apache.org/documentation) · kafka-python (kafka-python.readthedocs.io) · enunciado del laboratorio, CC3067.